

AR 技術を用いた大学キャンパスナビゲーションシステム 「IKU NAVI」の開発

—専修大学生田キャンパスにおける屋内外統合経路案内の実装と運用—

専修大学 ネットワーク情報学部 生亀プロジェクト

2026 年 9 月

概要

専修大学生田キャンパスは複数の建物が入り組んで配置されており、初めて訪れる学生・教職員・来訪者が目的の教室や施設に迷わずたどり着くことは容易ではない。本活動報告では、我々「生亀プロジェクト」が開発・運用している AR 対応キャンパスナビゲーション Web アプリケーション「IKU NAVI」について述べる。本システムは、CSV で管理された建物内外のノード・エッジデータをもとに Dijkstra 法による最短経路探索を行い、屋外は Google Maps、屋内は SVG フロアマップを用いて経路を可視化する。さらに、AR 領域では屋内は経路写真と方向矢印のオーバーレイ、屋外は Three.js とスマートフォンの GPS・ジャイロセンサを用いたリアルタイム AR によって、直感的なステップナビゲーションを実現している。インフラには Docker Swarm および cloudflare トンネルを用いた低コストかつスケーラブルな運用体制を構築した。本稿では要件定義からシステム設計、実装、運用に至る一連の開発プロセスと、そこから得られた知見、および今後の課題について報告する。

目次

1	はじめに	1
2	システム全体概要	2
2.1	基本機能	2
2.2	技術スタック	2
3	データ設計	2
3.1	ノード ID 体系	2
3.2	主要データファイル	3
4	経路探索アルゴリズム	3
4.1	グラフ構築	3
4.2	最短経路計算	4
4.3	屋内外接続と GPS 連携	4
5	フロントエンド UI	4
6	AR 実装	4
6.1	屋内 AR：経路写真と方向矢印のオーバーレイ	4
6.2	屋外 AR：Three.js + GPS + ジャイロセンサ	5
6.3	プライバシー・バッテリーへの配慮	5

7	インフラ構成	5
7.1	本番環境	5
7.2	Cloudflare Pages 移行	6
7.3	今後の静的データ配信への移行方針	6
8	開発プロセスと運用実績	6
9	今後の課題	6
10	おわりに	7

1 はじめに

専修大学生田キャンパスは、複数号館が渡り廊下や連絡通路で接続された複雑な構造を持ち、新入生や来訪者にとって目的の教室・食堂・トイレなどへの経路を把握することが困難な場合がある。従来の紙媒体の案内図や、館内に設置された固定の案内板だけでは、現在地から目的地までの動的な経路案内を提供することができない。

そこで我々「生亀プロジェクト」は、2026 年度ネットワーク情報学部活動として、スマートフォンから利用できる AR 対応キャンパスナビゲーションシステム「IKU NAVI」を企画・開発した。本プロジェクトの基本方針は、複雑な位置合わせや専用アプリのインストールを要する本格的な AR（拡張現実）ではなく、**まず経路写真に矢印を合成するというシンプルな形式の AR マップから着手し、段階的に発展させる**というものである。この方針のもと、2026 年 2 月のプロジェクト開始以降、データ設計・経路探索アルゴリズム・フロントエンド UI・インフラ構築までを一貫して自作し、本稿執筆時点（2026 年 9 月）で本番運用中である。

本稿では、第 2 章でシステム全体の概要を、第 3 章でデータ設計を、第 4 章で経路探索アルゴリズムを、第 5 章でフロントエンド UI を、第 6 章で AR 実装を、第 7 章でインフラ構成を、第 8 章で開発プロセスと運用実績を、第 9 章で今後の課題をそれぞれ述べる。

2 システム全体概要

IKU NAVI は、専修大学生田キャンパス内の経路をブラウザだけで案内する Web アプリケーションである。ネイティブアプリのインストールを必要とせず、スマートフォンや PC のブラウザから <https://iku-navi.net/> にアクセスするだけで利用できる。

2.1 基本機能

- 教室名・食堂名などのキーワード検索によるオートコンプリート付き目的地指定
- GPS による現在地取得（現在地から目的地までの自動ルーティング）
- 屋外区間は Google Maps 上に、屋内区間は SVG フロアマップ上に経路を重畳表示
- ステップ単位（矢印ボタン）でのナビゲーション進行
- AR 領域：屋内は経路写真＋方向矢印、屋外は Three.js によるリアルタイム AR 表示
- 最寄りのトイレ・食堂への経路探索

表 1 IKU NAVI 技術スタック一覧

区分	使用技術
バックエンド	Python 3 / Flask / Gunicorn / NetworkX / pandas
フロントエンド	HTML / CSS / Vanilla JavaScript / Google Maps API / Three.js / インライン SVG
インフラ	Docker Swarm / ConoHa VPS / Cloudflare Pages (静的配信)
ネットワーク	Cloudflare Tunnel (API 公開) / Cloudflare R2 (画像 CDN)
監視	Prometheus / Grafana / cAdvisor

2.2 技術スタック

システム全体のアーキテクチャは、静的コンテンツを Cloudflare Pages で配信し、経路探索 API を ConoHa VPS 上の Docker Swarm クラスタで稼働させる構成である。2026 年 7 月には、従来 Nginx が担っていた静的配信・振り分け機能を Cloudflare Pages と Cloudflare Tunnel のパスルーティングに置き換え、VPS 側の負荷を軽減する移行を行った。

3 データ設計

経路探索の基盤となるデータは、スプレッドシートで管理され、スクリプトによって CSV 形式で GitHub リポジトリに反映される運用となっている。

3.1 ノード ID 体系

建物内外のノードを単一の座標系・ID で扱うため、以下の規則でグローバル ID を付与している。

```

屋内ノード = building_id x 100,000 + local_id
屋外ノード = local_id + 9,000,000

```

各建物のローカル座標 (x, y, z) は、`anchors.csv` に記録された 2 点のアンカー座標から回転角 θ と平行移動量 (t_x, t_y, t_z) を自動算出し、以下の式でキャンパス全体のグローバル座標系へ変換される。

$$X_{global} = \cos \theta \cdot X_{local} - \sin \theta \cdot Y_{local} + t_x \quad (1)$$

$$Y_{global} = \sin \theta \cdot X_{local} + \cos \theta \cdot Y_{local} + t_y \quad (2)$$

$$Z_{global} = Z_{local} + t_z \quad (3)$$

アンカーが 1 点しか取得できない建物については、`buildings.json` に定義した `rot_deg` を回転角として用いる。

3.2 主要データファイル

エッジ種別は表 3 の 7 種類を定義しており、エスカレーターの一方向性やエレベーターの利用可否といった経路上の制約をデータ側で表現している。

なお、教室はノードそのものではなく**エッジの属性**として管理している点が本システムのデータ設計上の特徴である。これにより、1 つの教室が複数の出入口（ノード）を持つ場合でも自然に表現でき、経路探索時にはエッジの両端ノードを候補として全組み合わせを評価する方式を採っている（第 4 章）。

表 2 主要 CSV データファイル

ファイル	内容
node.csv (建物毎)	建物ローカル座標 (x,y,z), SVG 座標 (svg_x,svg_y), 階数, ノード種別
global_node.csv	屋外ノードの緯度経度 (lat,lng), グローバル座標
edge.csv (建物毎)	接続ノード対, 教室名 (; 区切りで複数対応), 探索コスト係数, 実距離, エッジ種別
edge_image.csv	エッジ両端ノードと CDN 上の経路画像ファイル名の対応 (約 999 行)
cafeteria_edge.csv	食堂の識別名・所属建物・UI 表示名
anchors.csv	建物ローカル座標とグローバル座標のアンカー対応点

表 3 エッジ種別一覧

type	意味	方向性
1	通路	双方向
2	階段	双方向
3	スロープ	双方向
4	エレベーター	双方向 (use_elevator=0 で除外可)
5	上りエスカレーター	一方向 (低い階→高い階のみ)
6	下りエスカレーター	一方向 (高い階→低い階のみ)
7	入口 (屋内外接続)	双方向 (入口ペナルティ +50 を加算)

4 経路探索アルゴリズム

4.1 グラフ構築

バックエンドは NetworkX の有向グラフ (DiGraph) を用いてキャンパス全体のグラフを構築する。エスカレーターのような一方向エッジはそのまま 1 方向のみ追加し、それ以外の通路・階段・エレベーター・入口については順方向・逆方向の両エッジを追加することで無向グラフと等価に扱っている。エレベーター利用の有無で経路が変わるため、起動時に「エレベーターあり」「エレベーターなし」の 2 種類のグラフを事前に構築してメモリにキャッシュし、切り替えコストをゼロにしている。

4.2 最短経路計算

経路探索には `networkx.bidirectional_dijkstra` を用いる。各エッジのコストは

$$\text{cost}(e) = \text{weight}(e) \times \text{length}(e)$$

で定義され、`weight` は経路種別ごとの通やすさを表す係数、`length` は実距離 (メートル) である。教室がエッジ属性として管理されているため、出発地・目的地がともに教室名で指定された場合は、それぞれの教室に対応するエッジの両端ノードを候補集合とし、全ての組み合わせについて `bidirectional_dijkstra` を実行して最小コストの経路を採用する。

4.3 屋内外接続と GPS 連携

`anchors.csv` に記録されたアンカーポイントをもとに、屋内ノードと屋外ノードを結ぶエッジを自動生成することで、建物をまたぐ経路探索を単一のグラフ上で完結させている。また、スマートフォンの GPS から取得し

た現在地は、Haversine 距離により最近傍の屋外ノードへスナップして出発ノードとする。GPS の精度が 30m を超える場合は案内板の参照を促す警告を表示し、最近傍ノードまでの距離が 500m (MAX_GPS_NODE_DIST_M) を超える場合はキャンパス外にいると判断して自動出発地点の採用を行わない。

5 フロントエンド UI

フロントエンドはフレームワークを用いず、Vanilla JavaScript で実装している。画面はレスポンス設計とし、768px 未満のスマートフォン表示では検索パネル・マップ・ナビゲーションバー・AR 領域を縦に積んだレイアウトに、768px 以上の PC・タブレット表示では左サイドバー（検索・ナビ・AR、幅 340px）と右側フルハイトのマップからなる 2 カラムレイアウトに切り替える。

経路上の各ノードが持つ `building` 属性の値によって、屋内外表示を自動的に切り替える。`building` が 0 であれば屋外と判定して Google Maps を表示し、進行方向に応じて `map.setHeading()` で地図を回転させる。0 以外であれば屋内と判定し、該当する SVG フロアマップを `fetch` してインライン挿入した上に、Dijkstra 法で求めた経路を重畳描画する。SVG 側は現在地までの区間を青色でハイライトし、進行方向を赤い三角矢印で示すとともに、経路の範囲から自動計算したビューボックスで表示領域を追従させる。

目的地検索には `datalist` 要素を用いず、独自実装のサジェストドロップダウンを採用している。クリック確定処理を `blur` イベントより先に発火させるため、選択肢の `mousedown` イベントで `preventDefault()` を呼び出し、クリックが無効化される典型的な不具合を回避している。

6 AR 実装

本システムの AR 表示は、ナビゲーション中の各ステップが屋内・屋外いずれのノードに属するかによって自動的に切り替わる、2 段階構成となっている。

6.1 屋内 AR：経路写真と方向矢印のオーバーレイ

屋内区間では、Cloudflare R2 上の CDN から配信される経路写真（両端ノードの組に対応する JPEG 画像）を表示し、その上に進行方向（左・右・直進）を示す矢印画像を重畳する。折れ角が 45 度以内の場合は直進と判定する。経路確定直後に全ステップ分の画像を `<img>` 要素として先読みし、DOM 上に積んでおくことで、ステップ移動時は表示クラスの付け替えのみで即座に画像を切り替えられるようにし、体感的な表示遅延を排除している。

6.2 屋外 AR：Three.js + GPS + ジャイロセンサ

屋外区間では、Three.js による WebGL レンダリングとリアカメラ映像を重ね合わせた本格的な AR を実装している。緯度経度は基準点からの相対距離（メートル）に変換した上で、ENU（東・上・北）に対応する右手系の Three.js 座標系にマッピングし、GPS 更新のたびに世界座標グループの位置を動かすことで、常に現在地を原点とする相対配置を実現している。

端末の方位センサーは機種・OS によって取得可能なイベントが異なるため、(1) `deviceorientationabsolute` (Android Chrome, 絶対方位)、(2) `webkitCompassHeading` (iOS Safari)、(3) `deviceorientation` の `alpha`（その他 Android, 相対方位）の優先順位でフォールバックする実装とした。また iOS 13 以降はセンサー利用に関するユーザー許可がタップ操作のコールスタック内でしか取得できない制約があるため、検索ボタン押下時点で許可ダイアログを先取りリクエストし、実際に AR 表示へ遷移する際の許可待ちを解消している。

6.3 プライバシー・バッテリーへの配慮

カメラおよび GPS の起動・解放は必要最小限に制御している。カメラは経路確定時に、ルート中に屋外 AR 区間が含まれる場合のみ先取り起動し、屋内のみの経路では一切起動しない。GPS の継続取得 (`watchPosition`) も屋外 AR 表示中のみ有効化する。ステップ移動のたびに「以降のステップに屋外 AR 区間が残っているか」を判定し、残っていなければカメラストリームの停止と位置情報取得の解除を行う。屋外区間に再突入した際は許可ダイアログを再表示せずにストリームを再取得する設計とし、プライバシーとバッテリー消費の両面に配慮した。

7 インフラ構成

7.1 本番環境

本番環境は ConoHa VPS（平常時メモリ 2GB・vCPU3 コア）上で Docker Swarm を用いて構築している。表 4 に主要サービスの構成を示す。学園祭などイベント開催時には 4GB サーバーを Swarm ワーカーとして追加増設し、Flask アプリのレプリカ数を 2 から 4 に拡張することで同時接続の増加に対応する運用としている。

表 4 Docker Swarm サービス構成

サービス	役割	平常時レプリカ
python	Flask + Gunicorn（経路探索 API）	2
counter	アクセスカウンター（Rails）	2
db	MariaDB 11	1（manager 固定）
cloudflared	Cloudflare Tunnel	1（manager 固定）
prometheus	メトリクス収集	1（manager 固定）
grafana	監視ダッシュボード	1（manager 固定）
cadvisor	コンテナリソース監視	全ノード

サーバーへは公開ポートを一切開放せず、Cloudflare Tunnel を介してのみ外部からアクセス可能とすることでファイアウォール設定の負担を軽減している。デプロイは cron（30 分毎）による `update.sh` の自動実行と、`start-first` によるローリングアップデートで無停止更新を実現している。

7.2 Cloudflare Pages 移行

2026 年 7 月、静的コンテンツ配信を担っていた Nginx を完全に撤去し、メインドメインを Cloudflare Pages、API ドメインを Cloudflare Tunnel 経由の直接配信に切り替える構成変更を行った。これにより、VPS 側で稼働させるサービスを Python・アクセスカウンター・DB・監視基盤・cloudflared のみに縮小し、2GB サーバーの負荷を軽減した。CORS は Flask 側の `after_request` で許可オリジンを制御し、QR コードなど既存導線との互換性は静的配信側のリダイレクト設定で担保している。

7.3 今後の静的データ配信への移行方針

現在は開発中のデータ追加・経路ロジック変更を 1 箇所（Flask バックエンド）に集約しておく方が変更コストが低いため、経路探索 API は Flask のまま運用している。キャンパス全体のグラフ規模は約 360 ノード・300 エッジと小さく、ブラウザ上の JavaScript でも Dijkstra 計算は 1 ミリ秒未満で完了する見込みであるため、全建物のデータ投入と経路ロジックの安定化が完了した段階で、グラフをビルド時に `graph.json` として静的出力し、経路計算をブラウザ側 JavaScript へ移行する計画である。この移行により、VPS 上の Python

コンテナと CORS 設定が不要となり、イベント時の同時接続増加に対しても静的配信の強みを活かせる見込みである。

8 開発プロセスと運用実績

本プロジェクトは 2026 年 2 月にリポジトリを開設して以降、本稿執筆時点（2026 年 9 月）までに 463 件のコミットを重ねてきた。開発体制は GitHub を用いたチーム開発とし、スプレッドシート上でのデータ編集を専用スクリプトによって自動的にコミット・プッシュする仕組みを整備することで、プログラミング経験の少ないメンバーでもデータ整備に参加できる体制を構築した。

開発補助ツールとして、全エッジ画像の CDN 上の存在確認を行う `Image_Checker`、全教室ペア間の経路を並列探索して異常な迂回・想定外の建物経由・フロアの逆走などを自動検出する `Route_Checker` を独自に開発し、データ追加時の品質担保に活用している。また、建物内座標のマッピング作業を支援する `Map_Editor` や、AR 用経路写真からの人物モザイク処理を行う `Human_Remover` など、データ整備・画像処理を効率化するツール群もあわせて内製した。

非機能要件としては、通常時 30 人以下・ピーク時（授業切替時）100 人以下の同時接続を想定し、静的ページは 500ms 以内、API レスポンスは 2 秒以内の応答を目標としている。可用性については大学内利用が主目的であることを踏まえ、高可用性クラスタ構成は採用せず、目標稼働率 70% 以上・Prometheus/Grafana によるリソース監視という現実的な水準に設定している。

9 今後の課題

本稿執筆時点で認識している主な課題は以下の通りである。

1. **屋内フロアマップデータの拡充**：現在 SVG フロアマップが整備済みなのは 1 建物分のみであり、ノード・エッジデータが存在する他の建物についても順次 SVG 化を進める必要がある。
2. **表示名データベース・イベントモードのデータ投入**：教室表示名や学園祭時の特別表示に関するテーブルはコードとして実装済みだが、実データが未投入のため機能していない。学園祭等の次回イベント前にデータ投入を完了させたい。
3. **自動テストの整備**：経路探索ロジック（エスカレーターの一方向制約・エレベーター除外・入口ペナルティ等）や `Route_Checker` の異常検出ロジックについて、現状は手動確認に依存しており、pytest 等による CI 組み込みが望まれる。
4. **屋内 AR の高度化**：現在の屋内 AR は経路写真＋矢印オーバーレイという簡易な形式にとどまっており、屋外 AR で確立したカメラ・GPS ライフサイクル管理の知見を活かしつつ、将来的には WebXR や CSS 3D 変換によるカメラ映像上への矢印重畳表示への発展を検討している。
5. **静的データ配信への移行**：第 7 章で述べた通り、経路探索のブラウザ側移行はデータ整備が一段落した後の重要なマイルストーンである。
6. **多言語対応・アクセシブルルート**：留学生やオープンキャンパス来場者向けの英語表示、および階段回避などのアクセシブルな経路提示についても検討課題として挙げられる。

10 おわりに

本活動報告では、専修大学生田キャンパスを対象とした AR 対応ナビゲーションシステム「IKU NAVI」について、企画の背景から、データ設計、経路探索アルゴリズム、フロントエンド UI、屋内外統合 AR 実装、そして Docker Swarm・Cloudflare を用いたインフラ構成に至るまでの開発内容を報告した。「まず写真に矢印

を合成するシンプルな形式の AR マップから始め、段階的に発展させる」という当初方針のもと、本システムは実際に大学構内で稼働するサービスとして実装・運用段階に至っている。今後はデータ整備の完了と自動テストの導入を優先課題としつつ、静的データ配信への移行や屋内 AR の高度化といった技術的发展にも取り組み、より多くの学生・来訪者にとって使いやすいキャンパスナビゲーションの実現を目指す。

参考文献

- [1] IKU NAVI 本番サイト, <https://iku-navi.net/>
- [2] 生亀プロジェクト GitHub リポジトリ, SenARMapProject_2026 (学内限定)